

게임 수학

# 강의 4: 벡터의 가위곱

외적(Cross Product) / 벡터 곱(Vector Product)

동명대학교 게임공학과  
강영민

2026년 1학기

## Contents

|     |                                 |    |
|-----|---------------------------------|----|
| 1   | 가위곱(외적) 개요                      | 2  |
| 2   | 가위곱의 정의                         | 2  |
| 2.1 | 기하적 정의                          | 2  |
| 2.2 | 대수적 정의 — 성분 계산                  | 3  |
| 3   | 가위곱의 크기와 기하적 의미                 | 3  |
| 3.1 | 크기 공식                           | 3  |
| 3.2 | 평행사변형의 넓이                       | 4  |
| 4   | 가위곱의 성질                         | 4  |
| 4.1 | 표준 기저 벡터의 가위곱                   | 5  |
| 5   | 벡터 삼중곱                          | 5  |
| 5.1 | 스칼라 삼중곱                         | 5  |
| 5.2 | 벡터 삼중곱 (BAC-CAB 공식)             | 6  |
| 6   | 내적 vs 가위곱 비교                    | 6  |
| 7   | 게임에서의 활용                        | 6  |
| 7.1 | 법선 벡터 계산                        | 6  |
| 7.2 | 방향 판별 (Orientation Test)        | 7  |
| 7.3 | 토크 (Torque) 계산                  | 7  |
| 7.4 | 각속도와 선속도                        | 7  |
| 8   | Python 구현                       | 7  |
| 9   | 실습 1: 가위곱 계산과 3D 시각화            | 9  |
| 9.1 | 수동 계산과 <code>np.cross</code> 비교 | 9  |
| 9.2 | 3D 시각화                          | 10 |
| 9.3 | 삼각형 면적 계산 — 세 꼭짓점에서의 가위곱        | 10 |
| 9.4 | 직교성 검증                          | 11 |

|                                             |           |
|---------------------------------------------|-----------|
| <b>10 실습 2: 법선벡터와 조명 (Diffuse Lighting)</b> | <b>12</b> |
| 10.1 삼각형의 법선벡터 시각화 . . . . .                | 12        |
| 10.2 확산 조명 (Diffuse Lighting) 공식 . . . . .  | 12        |
| 10.3 지형 메시(Terrain Mesh) 생성과 조명 . . . . .   | 13        |
| 10.4 2D 가위곱 . . . . .                       | 15        |
| <b>11 가위곱과 쿼터니언</b>                         | <b>15</b> |
| <b>12 연습 문제</b>                             | <b>15</b> |
| <b>13 요약</b>                                | <b>16</b> |
| <b>A 행렬식을 이용한 성분 공식 유도</b>                  | <b>16</b> |

# 1 가위곱(외적) 개요

지난 강의에서 다룬 내적(점곱)이 두 벡터의 유사도(similarity)나 정사영(projection)을 구하는 데 쓰인다면, 가위곱(cross product)은 두 벡터에 수직인 벡터를 구하는 연산이다. 내적이 스칼라를 반환하는 것과 달리, 가위곱은 벡터를 반환한다.

| 연산                  | 입력                                        | 출력                    | 주요 활용           |
|---------------------|-------------------------------------------|-----------------------|-----------------|
| 내적 (dot product)    | $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ | 스칼라 $\in \mathbb{R}$  | 각도, 투영, 유사도     |
| 가위곱 (cross product) | $\mathbf{u}, \mathbf{v} \in \mathbb{R}^3$ | 벡터 $\in \mathbb{R}^3$ | 법선벡터, 면적, 방향 판별 |

## [주의] 차원 제한

가위곱은 3차원 공간에서만 정의된다 (엄밀히는 7차원에서도 정의되지만 게임 수학에서는 3D만 다룬다). 2차원 벡터에서 가위곱을 사용하려면  $z = 0$ 으로 확장한 후  $z$  성분의 부호를 활용한다.

# 2 가위곱의 정의

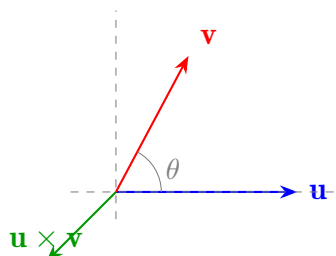
## 2.1 기하적 정의

### 정의 — 가위곱 (Cross Product)

$\mathbf{u}, \mathbf{v} \in \mathbb{R}^3$ 에 대해 가위곱  $\mathbf{w} = \mathbf{u} \times \mathbf{v}$ 는 다음 성질을 만족하는 벡터이다.

1. 방향:  $\mathbf{w}$ 는  $\mathbf{u}$ 와  $\mathbf{v}$  모두에 수직이다 ( $\mathbf{w} \perp \mathbf{u}, \mathbf{w} \perp \mathbf{v}$ ).
2. 크기:  $\|\mathbf{w}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$ , 여기서  $\theta$ 는 두 벡터 사이의 사잇각 ( $0 \leq \theta \leq \pi$ ).
3. 방향 결정: 오른손 법칙(right-hand rule)에 따른다.

**오른손 법칙(Right-Hand Rule).** 오른손의 네 손가락을  $\mathbf{u}$  방향에서  $\mathbf{v}$  방향으로 감아줄 때, 엄지손가락이 가리키는 방향이  $\mathbf{u} \times \mathbf{v}$ 의 방향이다.



## 2.2 대수적 정의 — 성분 계산

### 정의 — 가위곱 성분 공식

$\mathbf{u} = (u_1, u_2, u_3), \mathbf{v} = (v_1, v_2, v_3)$  일 때,

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix} = (u_2v_3 - u_3v_2, u_3v_1 - u_1v_3, u_1v_2 - u_2v_1)$$

여기서  $\mathbf{i} = (1, 0, 0), \mathbf{j} = (0, 1, 0), \mathbf{k} = (0, 0, 1)$  은 표준 기저 벡터이다.

행렬식 전개 설명. 위의  $3 \times 3$  행렬식을 첫 번째 행에 대해 여인수 전개(cofactor expansion) 하면:

$$\begin{aligned} \mathbf{u} \times \mathbf{v} &= \mathbf{i} \begin{vmatrix} u_2 & u_3 \\ v_2 & v_3 \end{vmatrix} - \mathbf{j} \begin{vmatrix} u_1 & u_3 \\ v_1 & v_3 \end{vmatrix} + \mathbf{k} \begin{vmatrix} u_1 & u_2 \\ v_1 & v_2 \end{vmatrix} \\ &= \mathbf{i}(u_2v_3 - u_3v_2) - \mathbf{j}(u_1v_3 - u_3v_1) + \mathbf{k}(u_1v_2 - u_2v_1) \end{aligned}$$

### 예제 1 — 가위곱 계산

$\mathbf{u} = (1, 2, 3), \mathbf{v} = (4, 5, 6)$  의 가위곱을 구하라.

풀이:

$$\begin{aligned} \mathbf{u} \times \mathbf{v} &= (u_2v_3 - u_3v_2, u_3v_1 - u_1v_3, u_1v_2 - u_2v_1) \\ &= (2 \cdot 6 - 3 \cdot 5, 3 \cdot 4 - 1 \cdot 6, 1 \cdot 5 - 2 \cdot 4) \\ &= (12 - 15, 12 - 6, 5 - 8) \\ &= (-3, 6, -3) \end{aligned}$$

검증:  $\mathbf{u} \cdot (\mathbf{u} \times \mathbf{v}) = 1(-3) + 2(6) + 3(-3) = -3 + 12 - 9 = 0 \checkmark$

$\mathbf{v} \cdot (\mathbf{u} \times \mathbf{v}) = 4(-3) + 5(6) + 6(-3) = -12 + 30 - 18 = 0 \checkmark$

## 3 가위곱의 크기와 기하적 의미

### 3.1 크기 공식

#### 정리 — 가위곱의 크기

$$\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$$

여기서  $\theta$  는  $\mathbf{u}$  와  $\mathbf{v}$  사이의 사잇각 ( $0 \leq \theta \leq \pi$ ).

크기와 내적의 비교.

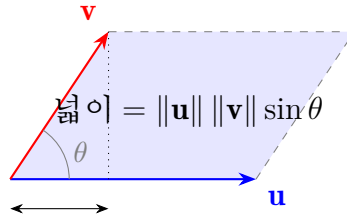
$$\text{내적: } \mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta, \quad \text{가위곱: } \|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$$

두 공식을 합치면 다음이 성립한다:

$$(\mathbf{u} \cdot \mathbf{v})^2 + \|\mathbf{u} \times \mathbf{v}\|^2 = \|\mathbf{u}\|^2 \|\mathbf{v}\|^2$$

### 3.2 평행사변형의 넓이

가위곱의 크기  $\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$  는 바로  $\mathbf{u}$  와  $\mathbf{v}$  를 두 변으로 하는 평행사변형의 넓이와 같다.



따라서 삼각형의 넓이는 평행사변형의 절반이므로:

$$S_{\Delta} = \frac{1}{2} \|\mathbf{u} \times \mathbf{v}\|$$

#### 예제 2 — 삼각형 넓이 계산

꼭짓점  $A = (1, 0, 0)$ ,  $B = (0, 1, 0)$ ,  $C = (0, 0, 1)$  인 삼각형의 넓이를 구하라.

$$\mathbf{AB} = B - A = (-1, 1, 0), \mathbf{AC} = C - A = (-1, 0, 1)$$

$$\mathbf{AB} \times \mathbf{AC} = (1 \cdot 1 - 0 \cdot 0, 0 \cdot (-1) - (-1) \cdot 1, (-1) \cdot 0 - 1 \cdot (-1)) = (1, 1, 1)$$

$$S_{\Delta} = \frac{1}{2} \|(1, 1, 1)\| = \frac{1}{2} \sqrt{1^2 + 1^2 + 1^2} = \frac{\sqrt{3}}{2} \approx 0.866$$

## 4 가위곱의 성질

#### 정리 — 가위곱의 대수적 성질

$\mathbf{u}, \mathbf{v}, \mathbf{w} \in \mathbb{R}^3, k \in \mathbb{R}$  에 대해:

1. 반교환성:  $\mathbf{u} \times \mathbf{v} = -(\mathbf{v} \times \mathbf{u})$
2. 분배법칙:  $\mathbf{u} \times (\mathbf{v} + \mathbf{w}) = \mathbf{u} \times \mathbf{v} + \mathbf{u} \times \mathbf{w}$
3. 스칼라 결합:  $(k\mathbf{u}) \times \mathbf{v} = k(\mathbf{u} \times \mathbf{v}) = \mathbf{u} \times (k\mathbf{v})$
4. 자기 가위곱:  $\mathbf{u} \times \mathbf{u} = \mathbf{0}$
5. 영벡터:  $\mathbf{u} \times \mathbf{0} = \mathbf{0}$
6. 평행 조건:  $\mathbf{u} \times \mathbf{v} = \mathbf{0} \Leftrightarrow \mathbf{u}$  와  $\mathbf{v}$  가 평행 ( $\mathbf{v} = k\mathbf{u}$ )
7. 교환법칙 성립 안 함:  $\mathbf{u} \times \mathbf{v} \neq \mathbf{v} \times \mathbf{u}$  (일반적으로)
8. 결합법칙 성립 안 함:  $(\mathbf{u} \times \mathbf{v}) \times \mathbf{w} \neq \mathbf{u} \times (\mathbf{v} \times \mathbf{w})$  (일반적으로)

**성질 1 직관.** 오른손 법칙에서  $\mathbf{u} \rightarrow \mathbf{v}$ 로 감으면 엄지가 위를 향하고,  $\mathbf{v} \rightarrow \mathbf{u}$ 로 감으면 엄지가 아래를 향한다. 따라서 방향이 반대  $\Rightarrow$  부호가 반전된다.

**성질 6 증명.**  $\mathbf{u} \times \mathbf{v} = \mathbf{0}$ 이면  $\|\mathbf{u}\| \|\mathbf{v}\| \sin \theta = 0$ .  $\mathbf{u}, \mathbf{v} \neq \mathbf{0}$  이라면  $\sin \theta = 0$ , 즉  $\theta = 0$  또는  $\theta = \pi$  이므로 평행.

## 4.1 표준 기저 벡터의 가위곱

| $\times$     | $\mathbf{i}$  | $\mathbf{j}$  | $\mathbf{k}$  |
|--------------|---------------|---------------|---------------|
| $\mathbf{i}$ | 0             | $\mathbf{k}$  | $-\mathbf{j}$ |
| $\mathbf{j}$ | $-\mathbf{k}$ | 0             | $\mathbf{i}$  |
| $\mathbf{k}$ | $\mathbf{j}$  | $-\mathbf{i}$ | 0             |

### 기억법

$\mathbf{i} \rightarrow \mathbf{j} \rightarrow \mathbf{k} \rightarrow \mathbf{i}$  순환 순서로 외적하면 양수, 역방향이면 음수:

$$\mathbf{i} \times \mathbf{j} = \mathbf{k}, \quad \mathbf{j} \times \mathbf{k} = \mathbf{i}, \quad \mathbf{k} \times \mathbf{i} = \mathbf{j}$$

## 5 벡터 삼중곱

### 5.1 스칼라 삼중곱

#### 정의 — 스칼라 삼중곱 (Scalar Triple Product)

세 벡터  $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$ 의 스칼라 삼중곱은:

$$\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \begin{vmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{vmatrix}$$

이 값의 절댓값은 세 벡터가 이루는 **평행육면체의 부피**와 같다.

스칼라 삼중곱의 성질:

- $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathbf{b} \cdot (\mathbf{c} \times \mathbf{a}) = \mathbf{c} \cdot (\mathbf{a} \times \mathbf{b})$  (순환 치환 불변)
- $\mathbf{a} \cdot (\mathbf{a} \times \mathbf{c}) = 0$  ( $\mathbf{a}$ 와  $\mathbf{a} \times \mathbf{c}$ 는 수직)
- 스칼라 삼중곱이 0이면 세 벡터는 같은 평면 위에 있다 (공면, coplanar).

## 5.2 벡터 삼중곱 (BAC-CAB 공식)

### 정리 — 벡터 삼중곱

$$\mathbf{a} \times (\mathbf{b} \times \mathbf{c}) = \mathbf{b}(\mathbf{a} \cdot \mathbf{c}) - \mathbf{c}(\mathbf{a} \cdot \mathbf{b})$$

이를 BAC-CAB 공식이라 부른다.

이 공식은 결합법칙이 성립하지 않음을 보여준다:  $(\mathbf{a} \times \mathbf{b}) \times \mathbf{c} \neq \mathbf{a} \times (\mathbf{b} \times \mathbf{c})$ .

## 6 내적 vs 가위곱 비교

| 특성    | 내적 ( $\mathbf{u} \cdot \mathbf{v}$ )                            | 가위곱 ( $\mathbf{u} \times \mathbf{v}$ )                         |
|-------|-----------------------------------------------------------------|----------------------------------------------------------------|
| 결과 타입 | 스칼라                                                             | 벡터                                                             |
| 기하 의미 | $\ \mathbf{u}\  \ \mathbf{v}\  \cos \theta$                     | $\mathbf{u}, \mathbf{v}$ 에 수직인 벡터                              |
| 크기    | —                                                               | $\ \mathbf{u}\  \ \mathbf{v}\  \sin \theta$                    |
| 교환법칙  | $\mathbf{u} \cdot \mathbf{v} = \mathbf{v} \cdot \mathbf{u}$     | $\mathbf{u} \times \mathbf{v} = -\mathbf{v} \times \mathbf{u}$ |
| 직교 조건 | $\mathbf{u} \cdot \mathbf{v} = 0$                               | $\mathbf{u} \times \mathbf{v} \neq \mathbf{0}$ (직교면 크기 최대)     |
| 평행 조건 | $ \mathbf{u} \cdot \mathbf{v}  = \ \mathbf{u}\  \ \mathbf{v}\ $ | $\mathbf{u} \times \mathbf{v} = \mathbf{0}$                    |
| 차원    | $n$ 차원 가능                                                       | 3차원만                                                           |

## 7 게임에서의 활용

### 7.1 법선 벡터 계산

3D 메시(mesh)의 각 삼각형 폴리곤에서 법선 벡터(surface normal)를 계산하는 것은 라이팅(조명), 충돌 처리, 백페이스 킬링(back-face culling) 등 다양한 곳에 쓰인다.

삼각형 꼭짓점  $P_0, P_1, P_2$ 가 주어지면:

$$\mathbf{n} = (P_1 - P_0) \times (P_2 - P_0)$$

정규화(normalize):

$$\hat{\mathbf{n}} = \frac{\mathbf{n}}{\|\mathbf{n}\|}$$

### 예제 3 — 삼각형 법선 벡터

꼭짓점  $P_0 = (0, 0, 0)$ ,  $P_1 = (1, 0, 0)$ ,  $P_2 = (0, 1, 0)$  인 삼각형의 법선 벡터를 구하라.

$$\mathbf{e}_1 = P_1 - P_0 = (1, 0, 0), \mathbf{e}_2 = P_2 - P_0 = (0, 1, 0)$$

$$\mathbf{n} = \mathbf{e}_1 \times \mathbf{e}_2 = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{vmatrix} = (0 \cdot 0 - 0 \cdot 1, 0 \cdot 0 - 1 \cdot 0, 1 \cdot 1 - 0 \cdot 0) = (0, 0, 1)$$

이미 단위 벡터이므로  $\hat{\mathbf{n}} = (0, 0, 1)$  (xy 평면의 법선 = z축 방향). ✓

## 7.2 방향 판별 (Orientation Test)

$\mathbf{u} \times \mathbf{v}$ 의 z 성분 부호를 이용하면 2D에서  $\mathbf{u}$ 에서  $\mathbf{v}$ 로의 회전 방향을 알 수 있다.

$$(\mathbf{u} \times \mathbf{v})_z = u_x v_y - u_y v_x \begin{cases} > 0 & \text{반시계 방향 (CCW)} \\ = 0 & \text{평행 (동일 방향 또는 반대)} \\ < 0 & \text{시계 방향 (CW)} \end{cases}$$

게임 활용 예시:

- 캐릭터가 목표물의 왼쪽/오른쪽에 있는지 판단
- 볼록 다각형(convex polygon) 내부 판별
- 삼각형 와인딩 순서(winding order) 확인 (백페이스 컬링)

### 예제 4 — 방향 판별

캐릭터 전방 벡터  $\mathbf{f} = (1, 0)$ , 적 방향 벡터  $\mathbf{e} = (0.5, 0.8)$ . 적이 캐릭터 기준 왼쪽인지 오른쪽인지 판별하라.

$$(\mathbf{f} \times \mathbf{e})_z = f_x e_y - f_y e_x = 1 \cdot 0.8 - 0 \cdot 0.5 = 0.8 > 0$$

⇒ 반시계 방향, 즉 적은 캐릭터 기준 왼쪽에 있다.

## 7.3 토크 (Torque) 계산

물리 시뮬레이션에서 토크(torque)는 회전력을 나타내며:

$$\boldsymbol{\tau} = \mathbf{r} \times \mathbf{F}$$

여기서  $\mathbf{r}$ 은 회전축에서 힘이 가해지는 점까지의 벡터,  $\mathbf{F}$ 는 힘 벡터.

## 7.4 각속도와 선속도

강체(rigid body)에서 각속도  $\boldsymbol{\omega}$ 와 임의 점의 선속도  $\mathbf{v}$ 의 관계:

$$\mathbf{v} = \boldsymbol{\omega} \times \mathbf{r}$$

여기서  $\mathbf{r}$ 은 회전 중심에서 해당 점까지의 벡터.

# 8 Python 구현



## 실습

```

1 import numpy as np
2
3 # 두벡터정의
4 u = np.array([1.0, 2.0, 3.0])
5 v = np.array([4.0, 5.0, 6.0])
6
7 # 가위곱 (NumPy 내장함수 )
8 cross = np.cross(u, v)
9 print("u x v =", cross)           # [-3.  6. -3.]
10
11 # 수동계산으로검증
12 def cross_product(a, b):
13     return np.array([
14         a[1]*b[2] - a[2]*b[1],
15         a[2]*b[0] - a[0]*b[2],
16         a[0]*b[1] - a[1]*b[0]
17     ])
18
19 cross_manual = cross_product(u, v)
20 print("수동 계산:", cross_manual)  # [-3.  6. -3.]
21
22 # 수직검증
23 print("u . (u x v) =", np.dot(u, cross)) # 0.0 수직()
24 print("v . (u x v) =", np.dot(v, cross)) # 0.0 수직()
25
26 # 크기검증
27 theta = np.arccos(np.dot(u, v) / (np.linalg.norm(u) * np.linalg.
28     norm(v)))
29 mag_formula = np.linalg.norm(u) * np.linalg.norm(v) * np.sin(
30     theta)
31 mag_actual = np.linalg.norm(cross)
32 print(f"크기 공식(): {mag_formula:.6f}")
33 print(f"크기 실제(): {mag_actual:.6f}")
34
35 # 반교환성검증
36 cross_vu = np.cross(v, u)
37 print("v x u =", cross_vu)           # [ 3. -6.  3.] = -(u x v)
38
39 # 법선벡터계산삼각형 ()
40 P0 = np.array([0.0, 0.0, 0.0])
41 P1 = np.array([1.0, 0.0, 0.0])
42 P2 = np.array([0.0, 1.0, 0.0])
43
44 e1 = P1 - P0
45 e2 = P2 - P0
46 normal = np.cross(e1, e2)
47 normal_unit = normal / np.linalg.norm(normal)
48 print("법선 벡터:", normal_unit)     # [0.  0.  1.]

```

```

48 # 2D 방향판별
49 def orientation_2d(forward, target):
50     """반환값
51     : > 0 => 왼쪽 (CCW), < 0 => 오른쪽 (CW), = 0 => 평행
52     """
53     return forward[0]*target[1] - forward[1]*target[0]
54
55 forward = np.array([1.0, 0.0])
56 enemy = np.array([0.5, 0.8])
57 result = orientation_2d(forward, enemy)
58 if result > 0:
59     print("적 위치: 왼쪽 (CCW)")
60 elif result < 0:
61     print("적 위치: 오른쪽 (CW)")
62 else:
63     print("적 위치: 정면평행 ()")
64
65 # 삼각형넓이
66 def triangle_area(A, B, C):
67     AB = B - A
68     AC = C - A
69     return 0.5 * np.linalg.norm(np.cross(AB, AC))
70
71 area = triangle_area(
72     np.array([1.0, 0.0, 0.0]),
73     np.array([0.0, 1.0, 0.0]),
74     np.array([0.0, 0.0, 1.0])
75 )
76 print(f"삼각형 넓이: {area:.6f}")      # sqrt(3)/2 ~= 0.866025

```

## 9 실습 1: 가위곱 계산과 3D 시각화

### 실습 환경

이 실습은 Google Colab에서 진행한다. 시각화를 위해 아래 코드로 visualizer 도구를 준비한다.

```

1 !wget https://raw.githubusercontent.com/dknife/linalg/main/tool/
   visualizer.py
2 from visualizer import *
3 !rm visualizer.py

```

### 9.1 수동 계산과 np.cross 비교

가위곱 성분 공식을 직접 코드로 구현하고 NumPy 결과와 비교한다.

### 실습 — 수동 가위곱 계산

```
1 import numpy as np
2
3 u = np.array([1, 2, 3])
4 v = np.array([2, 1, 2])
5
6 # 성분공식으로 직접 계산
7 #  $u \times v = (u_1 v_2 - u_2 v_1, u_2 v_0 - u_0 v_2, u_0 v_1 - u_1 v_0)$ 
8 uxv = np.array([
9     u[1]*v[2] - u[2]*v[1],
10    u[2]*v[0] - u[0]*v[2],
11    u[0]*v[1] - u[1]*v[0]
12 ])
13 print("수동 계산:", uxv)      # [ 1  4 -3]
14
15 # NumPy 내장함수로 검증
16 print("np.cross:", np.cross(u, v))  # [ 1  4 -3]
```

#### 주의: 부호 실수

$y$  성분 계산 시 부호에 주의하라. 공식은  $u_2 v_0 - u_0 v_2$  이다 ( $u_0 v_2 - u_2 v_0$  이 아니다). 행렬식 전개에서  $j$  앞에 음수 부호가 붙기 때문이다.

## 9.2 3D 시각화

가위곱 결과 벡터가 입력 두 벡터에 수직임을 시각적으로 확인한다.

### 실습 — 가위곱 3D 시각화

```
1 my3d = axis3d(x=[-10, 10], y=[-10, 10], z=[-10, 10])
2
3 u = np.array([4, 0, 0])
4 v = np.array([0, 2, 0])
5
6 draw_vec3d(my3d, u, color='r', label='u')
7 draw_vec3d(my3d, v, color='g', label='v')
8
9 uxv = np.cross(u, v)
10 draw_vec3d(my3d, uxv, color='b', label='uxv')
```

$\mathbf{u} = (4, 0, 0)$  과  $\mathbf{v} = (0, 2, 0)$  은  $xy$  평면 위의 벡터이므로, 가위곱  $\mathbf{u} \times \mathbf{v} = (0, 0, 8)$  은  $z$  축 방향이다.

## 9.3 삼각형 면적 계산 — 세 꼭짓점에서의 가위곱

삼각형  $ABC$ 의 면적을 어느 꼭짓점에서 계산하더라도 같은 결과가 나온다.

### 실습 — 삼각형 면적과 시각화

```
1 A = np.array([3, 5, 0])
2 B = np.array([-1, 2, 0])
3 C = np.array([5, -1, 0])
4
5 # 변벡터
6 AB = B - A
7 BC = C - B
8 CA = A - C
9
10 # 삼각형그리기
11 my3d = axis3d(x=[-10, 10], y=[-10, 10], z=[-10, 10])
12 draw_vec3d(my3d, AB, start_from=A, color='r', label='AB')
13 draw_vec3d(my3d, BC, start_from=B, color='r', label='BC')
14 draw_vec3d(my3d, CA, start_from=C, color='r', label='CA')
```

어느 꼭짓점을 기준으로 가위곱을 해도 평행사변형 면적(=삼각형 면적의 2배)이 동일하다:

### 실습 — 꼭짓점별 면적 계산

```
1 # 꼭짓점에서 B: BA x BC
2 BA = A - B; BC = C - B
3 cross_B = np.cross(BA, BC)
4 print("에서B:", np.linalg.norm(cross_B)) # 30.0
5
6 # 꼭짓점에서 C: CB x CA = (-BC) x CA
7 cross_C = np.cross(-BC, CA)
8 print("에서C:", np.linalg.norm(cross_C)) # 30.0
9
10 # 꼭짓점에서 A: AC x AB = (-CA) x AB
11 cross_A = np.cross(-CA, AB)
12 print("에서A:", np.linalg.norm(cross_A)) # 30.0
13
14 # 삼각형면적 = 평행사변형면적 / 2
15 print("삼각형 면적:", np.linalg.norm(cross_B) / 2) # 15.0
```

## 9.4 직교성 검증

가위곱 결과는 삼각형의 모든 변에 수직 (즉, 법선벡터)임을 내적으로 확인한다.

### 실습 — 내적으로 직교 확인

```
1 print(np.dot(cross_A, AB)) # 0
2 print(np.dot(cross_A, BC)) # 0
3 print(np.dot(cross_A, CA)) # 0
```

세 내적이 모두 0이므로 가위곱 벡터는 삼각형 평면에 수직이다.

## 10 실습 2: 법선벡터와 조명 (Diffuse Lighting)

가위곱의 대표적 응용인 법선벡터 기반 조명을 단계별로 구현한다. 이 과정은 게임 렌더링의 램버트 확산 조명(Lambertian diffuse lighting)의 핵심이다.

### 10.1 삼각형의 법선벡터 시각화

#### 실습 — 법선벡터 계산과 시각화

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 p0 = np.array([3.2, 1.1, 1.5])
5 p1 = np.array([2.2, 2.2, 1.0])
6 p2 = np.array([1.1, 1.2, 1.3])
7 triangle = np.array([p0, p1, p2])
8 color = np.array([1.0, 1.0, 0.0])
9
10 # 두변벡터
11 u = p1 - p0
12 v = p2 - p0
13
14 # 법선벡터 = 가위곱
15 N = np.cross(u, v)
16 center = (p0 + p1 + p2) / 3.0
17
18 mySpace = axis3d(x=[0, 4], y=[0, 4], z=[0, 4])
19 draw_polygons(mySpace, [triangle],
20               facecolors=[color], edgecolors='black', alpha=0.5)
21 draw_vec3d(mySpace, u, color='r', start_from=p0, label='u')
22 draw_vec3d(mySpace, v, color='r', start_from=p0, label='v')
23 draw_vec3d(mySpace, N, color='b', start_from=center, label='N')
24 setCam(mySpace, [1, 1, 1])
```

### 10.2 확산 조명 (Diffuse Lighting) 공식

광원 위치  $L_{\text{pos}}$ 와 삼각형 중심  $C$ 가 주어지면, 광원 방향 벡터  $\mathbf{L}$ 과 법선  $\hat{\mathbf{N}}$ 의 내적으로 밝기를 결정한다:

$$I = \max(\hat{\mathbf{N}} \cdot \hat{\mathbf{L}}, 0), \quad \hat{\mathbf{N}} = \frac{\mathbf{N}}{\|\mathbf{N}\|}, \quad \hat{\mathbf{L}} = \frac{L_{\text{pos}} - C}{\|L_{\text{pos}} - C\|}$$

$I > 0$ 이면 빛이 표면 앞쪽을 비추고,  $I \leq 0$ 이면 뒷면(그림자)이다.

#### 실습 — 광원 벡터와 조명 강도 계산

```
1 light_position = np.array([2.0, 2.0, 4.0])
2
3 # 법선벡터와광원벡터정규화
4 N = np.cross(u, v)
```

```

5 center = (p0 + p1 + p2) / 3.0
6 L = light_position - center
7
8 N = N / np.linalg.norm(N)
9 L = L / np.linalg.norm(L)
10
11 # 조명강도 = dot(N, L), 음수면 0
12 Intensity = N.dot(L)
13 if Intensity < 0:
14     Intensity = 0.0
15
16 # 강도에따라색상결정
17 color = np.array([1.0, 1.0, 0.0]) * Intensity
18
19 mySpace = axis3d(x=[0, 4], y=[0, 4], z=[0, 4])
20 draw_polygons(mySpace, [triangle],
21               facecolors=[color], edgecolors='black', alpha=0.5)
22 draw_vec3d(mySpace, N, color='b', start_from=center, label='N')
23 draw_vec3d(mySpace, L, color='b', start_from=center, label='L')
24 setCam(mySpace, [1, 1, 1])

```

### 10.3 지형 메시(Terrain Mesh) 생성과 조명

실제 게임에서는 수백~수만 개의 삼각형으로 지형이 구성된다. 여기서는  $15 \times 15$  격자를 만들고 면별(flat) 조명을 적용한다.

Step 1. 벡터스 격자 생성.

실습 — 벡터스 격자와 삼각형 메시

```

1 n_row, n_col = 15, 15
2 p = np.zeros((n_row, n_col, 3), dtype=float)
3 p[:, :, 1] = np.random.rand(n_row, n_col) * 0.2 # 축y 랜덤높이
4
5 xstep = 2 / n_col
6 zstep = 2 / n_row
7 for i in range(n_row):
8     for j in range(n_col):
9         p[i, j, 0] = j * xstep # x
10        p[i, j, 2] = i * zstep # z
11
12 # 삼각형리스트구성격자 (셀하나당삼각형개 2)
13 triangle_list = []
14 for i in range(n_row - 1):
15     for j in range(n_col - 1):
16         triangle_list.append([p[i+1][j], p[i][j+1], p[i][j]])
17         triangle_list.append([p[i+1][j], p[i+1][j+1], p[i][j+1]])
18

```

```

19 mySpace = axis3d(x=[0, 2], y=[0, 2], z=[0, 2])
20 draw_polygons(mySpace, triangle_list,
21               facecolors=['yellow'], edgecolors='black', alpha
                =0.5)

```

## Step 2. 면별 법선벡터 계산.

### 실습 — 면별 법선벡터

```

1 center_list = []
2 normal_list = []
3 for tri in triangle_list:
4     center_list.append((tri[0] + tri[1] + tri[2]) / 3.0)
5     u = tri[1] - tri[0]
6     v = tri[2] - tri[0]
7     N = np.cross(u, v)
8     N = N / np.linalg.norm(N)
9     normal_list.append(N)
10
11 mySpace = axis3d(x=[0, 2], y=[0, 2], z=[0, 2])
12 draw_polygons(mySpace, triangle_list,
13               facecolors=['yellow'], edgecolors='black', alpha
                =0.5)
14 for i in range(len(normal_list)):
15     draw_vec3d(mySpace, 0.2*normal_list[i],
16               color='b', start_from=center_list[i])
17 setCam(mySpace, [1, 1, 1])

```

## Step 3. 면별 확산 조명 적용.

### 실습 — 지형 메시에 조명 적용

```

1 light_vector = np.array([-1, 1, 0])
2 L = light_vector / np.linalg.norm(light_vector)
3 base_color = np.array([1.0, 1.0, 0.0])
4
5 color_list = []
6 for i in range(len(triangle_list)):
7     Intensity = L.dot(normal_list[i])
8     if Intensity < 0:
9         Intensity = 0
10    color_list.append(base_color * Intensity)
11
12 mySpace = axis3d(x=[0, 2], y=[0, 2], z=[0, 2])
13 draw_polygons(mySpace, triangle_list,
14               facecolors=color_list, edgecolors='black', alpha
                =0.5)
15 for i in range(len(triangle_list)):
16     draw_vec3d(mySpace, 0.2*normal_list[i],

```

17

```
color='b', start_from=center_list[i])
```

각 삼각형의 법선과 광원 방향의 내적( $\hat{\mathbf{N}} \cdot \hat{\mathbf{L}}$ )에 따라 밝기가 달라진다. 빛을 정면으로 받는 면은 밝고, 빛을 등지는 면은 어두워진다.

## 10.4 2D 가위곱

2차원 벡터의 가위곱은 스칼라 값을 반환하며, 평행사변형의 부호 있는 넓이에 해당한다.

### 실습 — 2D 가위곱

```
1 u = np.array([1, 2])
2 v = np.array([2, 0.5])
3
4 cross_2d = np.cross(u, v)
5 print(cross_2d) # -3.5
6
7 mySpace = axis2d(x=[0, 3], y=[0, 3])
8 draw_vec2d(mySpace, u, label='u')
9 draw_vec2d(mySpace, v, label='v')
```

결과가 음수이므로  $\mathbf{u}$ 에서  $\mathbf{v}$ 로의 회전은 시계 방향(CW)이다.

## 11 가위곱과 쿼터니언

가위곱은 쿼터니언(quaternion) 곱셈과 깊이 연결되어 있다. 순수 허수 쿼터니언  $p = (0, \mathbf{u})$ ,  $q = (0, \mathbf{v})$ 의 곱은:

$$pq = (-\mathbf{u} \cdot \mathbf{v}, \mathbf{u} \times \mathbf{v})$$

즉 실수부가 내적의 음수, 허수부가 가위곱이 된다. 이 관계는 강의 9(쿼터니언)에서 더 자세히 다룬다.

## 12 연습 문제

### 연습 문제

1.  $\mathbf{a} = (2, -1, 3)$ ,  $\mathbf{b} = (1, 4, -2)$  일 때  $\mathbf{a} \times \mathbf{b}$ 를 계산하고, 결과가  $\mathbf{a}$ 와  $\mathbf{b}$  모두에 수직임을 확인하라.
2. 꼭짓점이  $A = (1, 0, 0)$ ,  $B = (0, 2, 0)$ ,  $C = (0, 0, 3)$ 인 삼각형의 넓이를 구하라.
3. 다음을 증명하라:  $\|\mathbf{u} \times \mathbf{v}\|^2 + (\mathbf{u} \cdot \mathbf{v})^2 = \|\mathbf{u}\|^2 \|\mathbf{v}\|^2$  (힌트:  $\sin^2 \theta + \cos^2 \theta = 1$ 을 이용)
4. 2D 게임에서 플레이어 전방 방향  $\mathbf{f} = (0, 1)$ , 목표물 방향  $\mathbf{t} = (0.707, 0.707)$  일 때, 목표물이 왼쪽인지 오른쪽인지 판별하고 회전 각도를 구하라.
5.  $\mathbf{a} = (1, 1, 0)$ ,  $\mathbf{b} = (1, 0, 1)$ ,  $\mathbf{c} = (0, 1, 1)$ 의 스칼라 삼중곱을 구하고, 이 세 벡터가 같은 평면에 있는지 확인하라.



6. Python `numpy`를 사용하여 문제 1-5를 코드로 구현하고 결과를 검증하라.

## 13 요약

### 핵심 정리

1. 가위곱  $\mathbf{u} \times \mathbf{v}$ 는  $\mathbf{u}$ 와  $\mathbf{v}$  모두에 수직인 벡터를 반환한다.
2. 방향은 오른손 법칙으로 결정되며,  $\mathbf{u} \times \mathbf{v} = -(\mathbf{v} \times \mathbf{u})$  (반교환성).
3. 크기  $\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$ 는 평행사변형 넓이와 같다.
4. 성분 공식:  $(u_2v_3 - u_3v_2, u_3v_1 - u_1v_3, u_1v_2 - u_2v_1)$
5. 활용: 법선 벡터, 면적 계산, 방향 판별, 토크, 쿼터니언.
6. Python에서는 `numpy.cross(u, v)`로 계산한다.

## A 행렬식을 이용한 성분 공식 유도

$\mathbf{u} = u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}$ ,  $\mathbf{v} = v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}$ 로 쓰고 분배법칙을 적용하면:

$$\begin{aligned}\mathbf{u} \times \mathbf{v} &= (u_1\mathbf{i} + u_2\mathbf{j} + u_3\mathbf{k}) \times (v_1\mathbf{i} + v_2\mathbf{j} + v_3\mathbf{k}) \\ &= u_1v_1(\mathbf{i} \times \mathbf{i}) + u_1v_2(\mathbf{i} \times \mathbf{j}) + u_1v_3(\mathbf{i} \times \mathbf{k}) \\ &\quad + u_2v_1(\mathbf{j} \times \mathbf{i}) + u_2v_2(\mathbf{j} \times \mathbf{j}) + u_2v_3(\mathbf{j} \times \mathbf{k}) \\ &\quad + u_3v_1(\mathbf{k} \times \mathbf{i}) + u_3v_2(\mathbf{k} \times \mathbf{j}) + u_3v_3(\mathbf{k} \times \mathbf{k})\end{aligned}$$

표준 기저의 가위곱 결과 ( $\mathbf{i} \times \mathbf{i} = \mathbf{0}$ ,  $\mathbf{i} \times \mathbf{j} = \mathbf{k}$  등)를 대입하면:

$$\begin{aligned}&= u_1v_2\mathbf{k} - u_1v_3\mathbf{j} - u_2v_1\mathbf{k} + u_2v_3\mathbf{i} + u_3v_1\mathbf{j} - u_3v_2\mathbf{i} \\ &= (u_2v_3 - u_3v_2)\mathbf{i} - (u_1v_3 - u_3v_1)\mathbf{j} + (u_1v_2 - u_2v_1)\mathbf{k}\end{aligned}$$

이것이 바로 행렬식 전개 결과와 일치한다. □